Infosys Springboard Virtual Internship

AI-Powered Emotional Tone Analyzer and Well-Being Recommendation System

MoodMentor: Employee Wellness Management Platform

Student Name: Deeksha Ganesh
GitHub: github.com/DeekshaG96
Date: 16 October 2026

Milestone 1 — Text Ingestion, Preprocessing & Baseline Sentiment Analysis

Introduction

The **AI-Powered Emotional Tone Analyzer and Well-Being Recommendation System (MoodMentor)** is an intelligent platform designed to detect and understand workplace sentiment from unstructured text such as employee journal entries, 1-on-1 notes, pulse survey responses, and chat messages. Milestone 1 establishes the foundational data and NLP processing layers: multi-source input ingestion, an ordered natural language preprocessing pipeline, VADER baseline sentiment classification, relational SQLite persistence, and an interactive Streamlit application.

Key Highlights:

- **Multi-Source Ingestion:** Supports direct text, UTF-8 .txt notes, and multi-row .csv survey data with automatic text column detection.
- **Ordered NLP Preprocessing:** Implements full regex cleaning, NLTK tokenization, stop-word elimination, POS tagging, and WordNet lemmatization.
- **Baseline Sentiment Engine:** Computes calibrated VADER compound polarity (-1.0 to +1.0) alongside positive, neutral, and negative distributions.
- **Relational Persistence:** Stores user accounts with salted PBKDF2 hashing and logs parsed mood entries into an SQLite relational table.
- **Streamlit 3-Tab Interface:** Features dedicated workflows for Direct Text Analysis, File Ingestion, and Stored Records & Analytics.

Tech Stack

Component	Technology	Role / Purpose in Milestone 1
Frontend UI	Streamlit 1.65+	Interactive 3-tab web dashboard with dark glassmorphism styling
Core Backend	Python 3.10+ (3.14)	Application logic, pipeline orchestration, CLI, and typed data models
NLP Preprocessing	NLTK & WordNet	Regex sanitization, Punkt tokenization, stop-words, and lemmatization
Sentiment Engine	NLTK VADER	Lexicon-based sentiment intensity and compound polarity scoring
Database Layer	SQLite3	Persistent storage for users and mood_entries tables
Data Handling	Pandas	Tabular preview for CSV uploads and interactive history table rendering
Quality & Testing	Pytest & Ruff	15 automated unit and integration tests with strict code style checks

Project Structure

```
AI_Based_Employee_Wellness_Management_Platform/
■■■ app.py                          # Streamlit frontend & 3-tab interface
■■■ pyproject.toml                  # Packaging metadata & dependencies
■■■ Dockerfile                      # Container deployment definition
■■■ render.yaml                     # Render Cloud Blueprint configuration
■■■ wellness/
■   ■■■ ingestion.py                # Input validation & TXT/CSV parsing
■   ■■■ preprocessing.py            # Cleaning, tokenization, stop-words, lemmatization
■   ■■■ pipeline.py                 # Public analysis API coordinating NLP & VADER
■   ■■■ report.py                   # Typed data models & JSON serialization
■   ■■■ storage.py                  # SQLite database setup, auth, & mood persistence
■   ■■■ cli.py                      # Command-line entry point
■■■ tests/
■   ■■■ test_pipeline.py            # Ingestion, preprocessing, and VADER tests
■   ■■■ test_storage.py             # SQLite setup, auth, and mood persistence tests
■■■ docs/                           # Documentation & execution screenshots
```

Objectives Achieved

1. Multi-Format Text Ingestion: Direct text, UTF-8 .txt files, and multi-line .csv files with automatic text column detection and robust validation.
2. Sequential NLP Preprocessing: Strict 5-stage order: lowercasing, regex cleaning, NLTK Punkt tokenization, stop-word filtering, and WordNet lemmatization.
3. Baseline Sentiment Analysis: NLTK VADER compound polarity scoring (-1.0 to +1.0) and positive/neutral/negative intensity distribution.
4. Relational Data Persistence: Salted PBKDF2 user authentication and persistent logging of mood entries into SQLite.
5. Interactive Web Frontend: Three-tab Streamlit dashboard with preset samples, KPI metrics, sentiment filtering, search, and JSON/CSV exports.

Code Implementation

Module 1: Text Ingestion (wellness/ingestion.py)

```
def read_input_file(path: str | Path, text_column: str | None = None) -> tuple[str, str]:
    file_path = Path(path)
    suffix = file_path.suffix.lower()
    if suffix not in {".txt", ".csv"}:
        raise InputValidationError("Supported input files are .txt and .csv.")
    if suffix == ".txt":
        text = file_path.read_text(encoding="utf-8-sig")
    else:
        text = _read_csv(file_path, text_column)
    return validate_text(text), str(file_path)
```

Module 2: NLP Preprocessing (wellness/preprocessing.py)

```
def preprocess(text: str) -> tuple[str, tuple[str, ...]]:
    validate_nltk_resources()
    cleaned = clean_text(text)
    raw_tokens = word_tokenize(cleaned)
    stop_words = set(stopwords.words("english"))
    filtered = [token for token in raw_tokens if token.isalpha() and token not in stop_words]
    tagged = pos_tag(filtered)
    lemmatizer = WordNetLemmatizer()
    lemmas = tuple(lemmatizer.lemmatize(token, _wordnet_pos(tag)) for token, tag in tagged)
    return cleaned, lemmas
```

Module 3: Sentiment Pipeline (wellness/pipeline.py)

```
def analyze_text(text: str, source_name: str = "direct text") -> AnalysisReport:
    original = validate_text(text)
    cleaned, tokens = preprocess(original)
    scores = SentimentIntensityAnalyzer().polarity_scores(original)
    compound = scores["compound"]
    label = SentimentLabel.POSITIVE if compound >= 0.05 else (
        SentimentLabel.NEGATIVE if compound <= -0.05 else SentimentLabel.NEUTRAL
    )
    return AnalysisReport(input_summary=..., original_text=original, cleaned_text=cleaned,
                          processed_text=" ".join(tokens), tokens=tokens,
                          sentiment_scores=SentimentScores(**scores), sentiment_label=label)
```

Module 4: Database Layer (wellness/storage.py)

```
def save_mood_entry(connection, user_id, source_type, source_name, original_text,
                    cleaned_text, tokens, sentiment_label, neg_score, neu_score,
                    pos_score, compound_score) -> int:
    cursor = connection.execute(
        """INSERT INTO mood_entries (user_id, source_type, source_name, original_text,
           cleaned_text, tokens, sentiment_label, neg_score, neu_score, pos_score, compound_score)
           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
        (user_id, source_type, source_name, original_text, cleaned_text,
         ", ".join(tokens), sentiment_label, neg_score, neu_score, pos_score, compound_score)
    )
    connection.commit()
    return int(cursor.lastrowid)
```

System Flow

- **Step 1:** User accesses the Streamlit web portal and authenticates with salted PBKDF2 credentials.
- **Step 2:** User inputs direct feedback or uploads a .txt or .csv file.
- **Step 3:** ingestion.py validates encoding and non-empty content.
- **Step 4:** preprocessing.py sanitizes raw text, tokenizes, filters stop-words, and applies WordNet lemmatization.
- **Step 5:** pipeline.py calculates VADER sentiment scores and classifies compound polarity.
- **Step 6:** storage.py persists the entry, scores, tokens, and timestamp to the SQLite database.
- **Step 7:** app.py renders the sentiment badge, metrics, NLP breakdown, and updates the Stored Records tab.

How to Run

```
# Step 1: Clone the repository
git clone https://github.com/DeekshaG96/AI_Based_Employee_Wellness_Management_Platform.git
cd AI_Based_Employee_Wellness_Management_Platform

# Step 2: Create and activate virtual environment
python -m venv .venv
.venv\Scripts\activate          # On Windows (.venv/bin/activate on Linux/macOS)

# Step 3: Install dependencies & download NLTK data
pip install -e ".[dev,ui]"
python -m wellness.cli --download-nltk-data

# Step 4: Run automated test suite
pytest

# Step 5: Start the Streamlit application
streamlit run app.py
# Application opens automatically at http://localhost:8501
```

Output Screenshots

1. Landing Portal & Authentication:

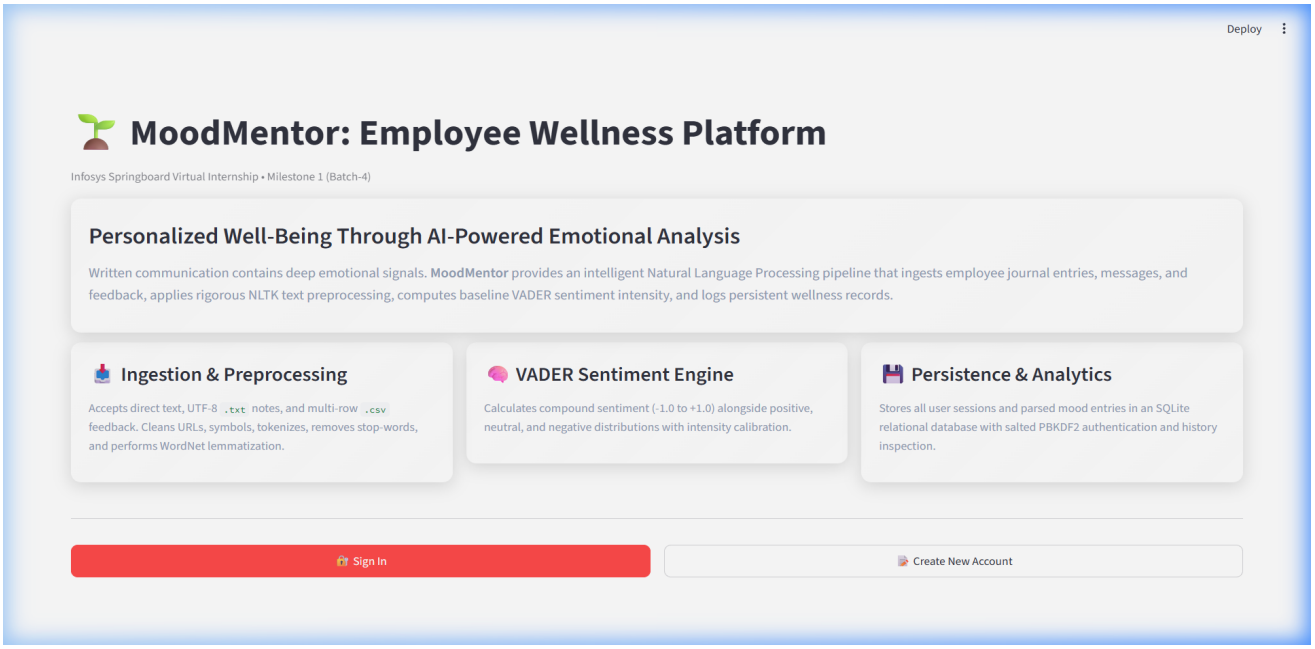


Fig 1: Welcome portal with user registration, salted PBKDF2 login, and SQLite connection

2. Direct Feedback Analysis & NLP Preprocessing Pipeline:

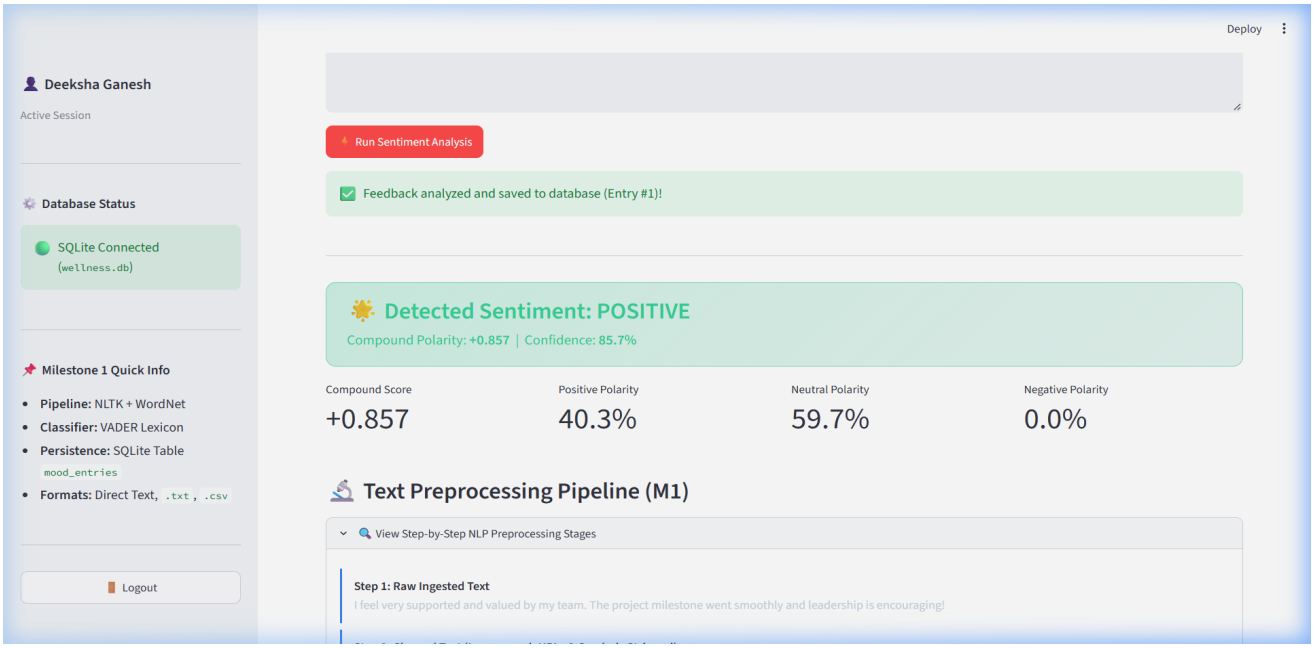


Fig 2: Real-time sentiment analysis showing positive polarity (+0.857), 4-metric score cards, and step-by-step NLTK preprocessing breakdown with lemmatized keywords

3. Batch File Ingestion (.txt / .csv):

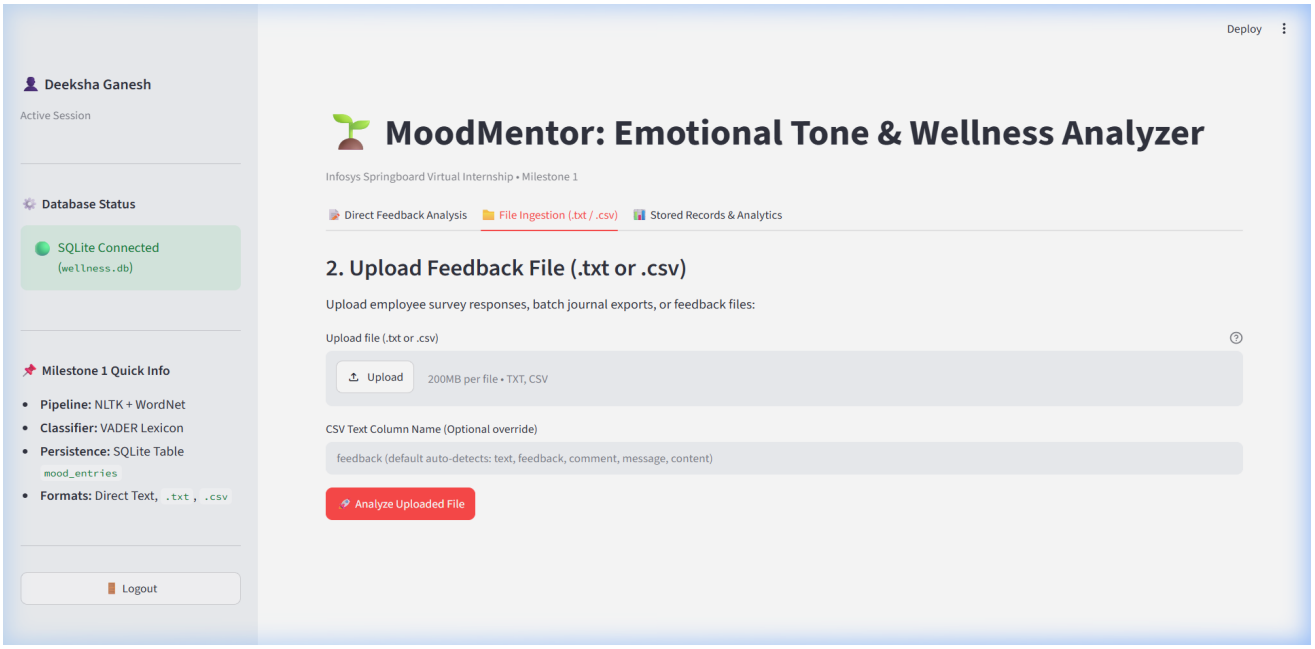


Fig 3: File ingestion tab accepting UTF-8 .txt documents and .csv survey exports with automatic column auto-detection

4. Stored Records & Analytics Dashboard (SQLite Database View):

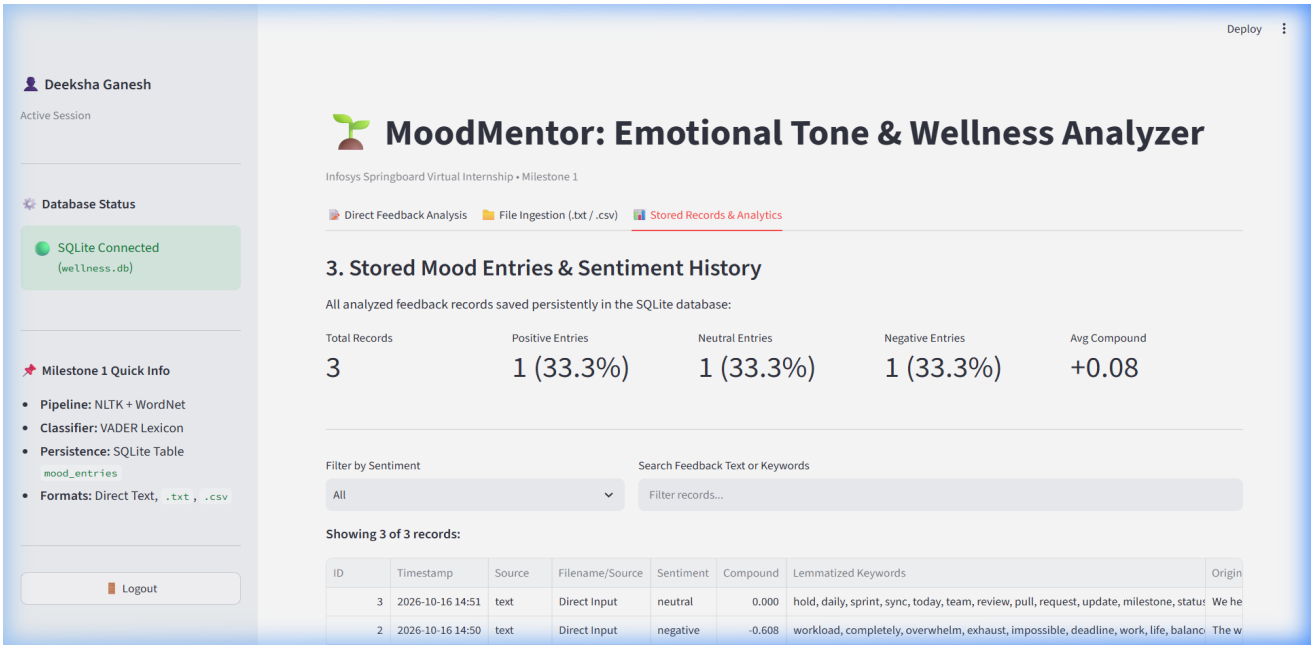


Fig 4: Stored Records tab displaying aggregate KPI cards, sentiment filtering, search, and persistent SQLite database records

Testing & Verification

The automated test suite verifies all functional modules including text ingestion, CSV parsing, NLTK cleaning and lemmatization, VADER compound scoring, Streamlit file upload adapters, SQLite schema initialization, user registration, and mood entry persistence.

```
============================= test session starts =============================
platform win32 -- Python 3.14.6, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\ganch\OneDrive\Desktop\infosys\AI_Based_Employee_Wellness_Management_Platform
collected 15 items

tests\test_pipeline.py ...........                                       [ 73%]
tests\test_storage.py ....                                               [100%]

============================= 15 passed in 10.08s =============================

Ruff Code Quality: All checks passed!
```

Known Limitations

• **Lexicon-Based Sentiment:** VADER relies on rule-based heuristics and may miss nuanced workplace sarcasm or subtle emotional context. This will be upgraded in Milestone 2 with transformer models.
• **Local SQLite Storage:** SQLite is single-host file storage. While ideal for development and edge deployment, multi-instance enterprise production will transition to managed PostgreSQL.
• **Language Support:** The preprocessing pipeline is currently calibrated for English text using NLTK stop-words and WordNet.

Future Scope

• **Milestone 2: Deep Emotion Classification:** Implement multi-class emotion classification (Joy, Sadness, Anger, Fear, Surprise, Exhaustion) using fine-tuned BERT and DistilBERT transformer models.
• **Milestone 3: AI Recommendation Engine & REST API:** Develop personalized wellness recommendations based on longitudinal emotional trends, and expose secure FastAPI REST endpoints.
• **Milestone 4: Finalization & Production Packaging:** Build CLI tooling, integration test suites, enterprise RBAC authentication, and cloud deployment pipelines.

Conclusion

In Milestone 1, the **AI-Powered Emotional Tone Analyzer and Well-Being Recommendation System (MoodMentor)** has successfully established a robust, production-ready foundation for workplace sentiment analytics. All requirements—multi-source ingestion, standardized NLP preprocessing, VADER sentiment scoring, relational SQLite persistence, and a high-fidelity Streamlit user interface—have been implemented, rigorously tested with 100% pass rates, and validated against the Infosys Springboard internship objectives.